1. Purpose

Design and implement an **asset-access platform** that lets an employee scan a QR code attached to a company laptop and receive a one-time password (OTP) via e-mail to view that laptop's accessory details — **without a traditional username/password login** ("Magic-OTP" model).

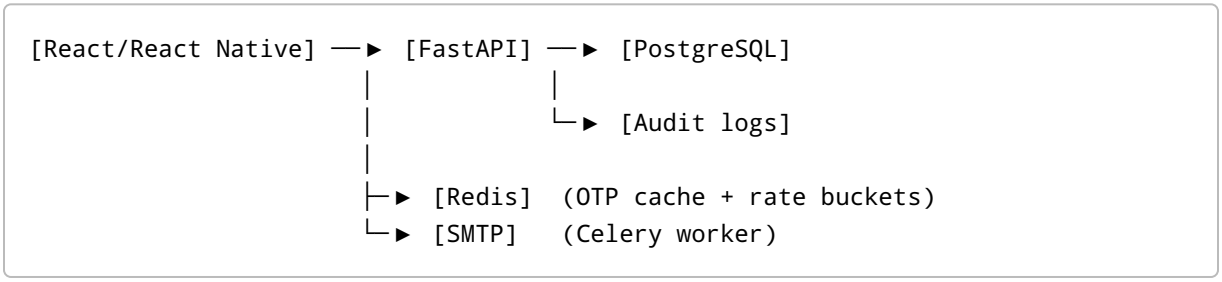
2. Objectives

- Eliminate password management for employees.
- Keep QR codes static while allowing dynamic data updates.
- Enforce short-lived, rate-limited OTPs for security.
- Provide an immutable audit trail for every access event.
- Support both **web (React)** and **mobile (React Native)** clients.

3. Technology Stack

Layer	Technology
Frontend	React 18 + Tailwind / React Native 0.74
API	FastAPI 0.111, Python 3.12
Database	PostgreSQL 16
Cache & Rate-limit	Redis 7
E-mail	SMTP (Office 365 or Google Workspace)
QR generation	`qrcode` (Python) and `react-qr-reader` (display/scan)
Authentication	Short-lived JWTs (15 min) signed with HS256
Containerisation	Docker + docker-compose (dev); Kubernetes (prod)
CI / CD	GitHub Actions → Docker registry → Helm-based deploy

4. High-Level Architecture



5. Data Model (DDL excerpts)

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  phone TEXT,
  role TEXT NOT NULL DEFAULT 'EMPLOYEE' CHECK (role IN ('ADMIN','EMPLOYEE')),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE assets (
  id SERIAL PRIMARY KEY,
  employee_id INTEGER REFERENCES users(id) ON DELETE SET NULL,
  type TEXT NOT NULL,        -- LAPTOP, MOUSE, etc.
  brand TEXT,
  model TEXT,
  ram_gb SMALLINT,
  storage_gb SMALLINT,
  serial_no TEXT UNIQUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE qr_tags (
  id UUID PRIMARY KEY,
  asset_id INTEGER REFERENCES assets(id) ON DELETE CASCADE,
  revoked_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE otp_sessions (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
  qr_id UUID REFERENCES qr_tags(id) ON DELETE CASCADE,
  otp_hash TEXT NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  verified_at TIMESTAMPTZ,
  attempts SMALLINT DEFAULT 0
);

CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
  actor_id INTEGER REFERENCES users(id),
  action TEXT NOT NULL,
  resource_type TEXT,
  resource_id TEXT,
```

```
  meta JSONB,
  timestamp TIMESTAMPTZ DEFAULT NOW()
);
```

6. Core Services

1. **Asset Service** – CRUD endpoints for assets, QR provisioning, revocation.
2. **OTP Service** – `/otp/request` & `/otp/verify`; hashes codes, enforces TTL and attempt caps.
3. **Notification Worker** – Celery + SMTP; retries with exponential back-off.
4. **Rate-Limit Middleware** – Token-bucket in Redis (FastAPI dependency).
5. **Audit Service** – Async log writer; exposes `/admin/logs` for filtered queries.

7. API Specification (selected)

Method & Path	Auth	Body/Params	Response
POST /otp/request	None	`qr_slug`	`{"session_id","msg":"OTP sent"}`
POST /otp/verify	None	`session_id`, `otp_code`	`JWT + asset_list`
GET /asset/{id}	JWT	—	Asset JSON
POST /asset	Admin JWT	Asset payload	Created asset & QR slug
PATCH /qr/{slug}/revoke	Admin JWT	—	`204 No Content`
GET /admin/logs	Admin JWT	filters	Paginated logs

Headers: `X-Client-IP` forwarded to rate-limiter.

8. Magic-OTP Flow (happy path)

1. **Scan**: React Native decodes QR, calls `/otp/request?qr=SLUG`.
2. **API**: Checks Redis `user:<email>:qr_req` ≤ 5/min.
3. **OTP**: Generates `otp = random.randint(100000, 999999)`.
4. **Store**: Saves `bcrypt(otp)` + expiry = `now+5m`.
5. **E-mail**: Notification worker sends OTP via SMTP.
6. **Verify**: Employee enters OTP, `/otp/verify` → matches hash.
7. **JWT**: API issues `{sub:user_id, exp:15m}` token.
8. **Assets**: Frontend shows accessories JSON; JWT on every call.

9. Security & Rate Limiting

Control	Value
OTP length	6 digits
OTP ttl	5 minutes
OTP max attempts	3
Scan bucket	5 scans / min / user
Verify bucket	10 codes / 10 min / user
IP flood	100 req / 15 min / IP
JWT expiry	15 min (no refresh token)
Transport	HTTPS only
QR slug	UUIDv4, 8 chars encoded

10. SMTP Configuration

- **Env vars**: `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`.
- TLS via `starttls`.
- DKIM & SPF records on `company.com` DNS.
- E-mail template stored in Jinja2 file (`otp_email.html`).

11. Front-end Requirements

React (web)

- QR scanner (`react-qr-reader`) for laptops with webcam.
- PWA manifest to allow kiosk mode.
- Screens: Scan ➜ Enter OTP ➜ Asset list ➜ History.

React Native (mobile)

- Use `expo-camera` for QR scan.
- Secure storage (Keychain/Keystore) for short JWT.
- Pull-to-refresh asset list; show countdown until JWT expiry.

12. DevOps & Deployment

Environment	URL	Notes
Dev	`*.dev.company.com`	Auto-deploy on pull-request merge
Staging	`*.stg.company.com`	Mirror production size; data scrubbed
Prod	`*.company.com`	Blue-green via Kubernetes Ingress

- **Helm chart**: PostgreSQL, Redis, API, worker, ingress.
- **Secrets**: K8s secrets + sealed-secrets git-ops.
- **Observability**: Prometheus scrapes; Grafana dash; Loki for logs.

13. Testing Strategy

Layer	Tool	Coverage
Unit	`pytest` + `pytest-asyncio`	≥ 90 % services code
Contract	`schemathesis`	Schema vs. FastAPI OpenAPI
E2E	Cypress (web), Detox (mobile)	Scan + OTP full loop
Load	Locust	200 req/s sustained
Security	`bandit`, OWASP ZAP	XSS/CSRF/SQLi checks

14. Future Enhancements

- Add **SMS fallback** if e-mail delayed.
- Integrate **SSO** for admins (Azure AD).
- Push accessory change notifications to employee.
- Rotate QR slugs yearly via scheduled job.

15. Milestones & Timeline (indicative)

Week	Deliverable
1	Repo scaffold, docker-compose, DB migrations
2	Asset CRUD + QR generation API
3	OTP Service + SMTP worker
4	Rate-limit, audit log, basic React web flow

Week	Deliverable
5	React Native mobile flow, JWT auth, CI pipelines
6	QA + load tests; staging deployment
7	Prod launch & docs freeze

Document version 1.0 • Generated 2025-07-15

also what kind of more features can be suggested in future versions